

Lecture 11 Q&A

Recursion, printReverse, and DFS: your questions from
the slip

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 11 Tuesday, October 6, 2026

What You Asked

Thank you for the thoughtful questions on the lecture slip. They fall into six groups:

Theme	Where we answer it
More examples; <code>printReverse</code>	Stack traces (" <code>abc</code> ", <code>digitSum</code>), forward vs. backward, helpers, a recipe
DFS again, in general and in more detail	DFS in one slide, then the slip maze step by step
Slip question 3	Trace, decision table, call tree, answers
Left/right/up/down order; <code>unmark</code> and <code>remove</code> ?	Does the order matter? What is undone when we backtrack?
BFS vs. DFS; queues; counting cells	Comparison; queue, <code>enqueue</code> , <code>dequeue</code> ; size of a circular queue (<code>front_</code> , <code>back_</code>)
<code>size_type</code> ; reviewing	Short slides near the end

More printReverse Examples: Trace "abc"

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }           // base case  
    printReverse(s.substr(1));           // smaller: rest of s  
    std::cout << s[0];                  // combine: then s[0]  
}
```

Call	Calls with	Prints on return	Order
("abc")	"bc"	a	3rd
("bc")	"c"	b	2nd
("c")	" "	c	1st
("")	(base case)	nothing	—

Read it bottom-up

The calls go down: "abc", "bc", "c", "". The printing happens on the way *back up*, so the last character is printed first: cba.

Swap Two Lines: Forward Instead of Backward

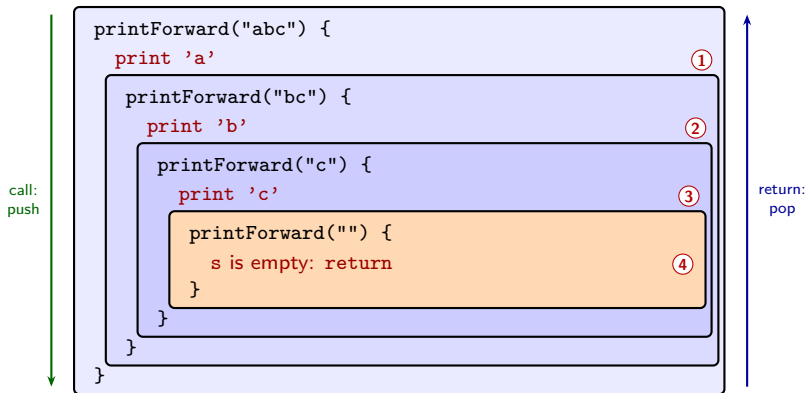
Only the position of the print changes.

```
// prints "abc"
void printForward(std::string s) {
    if (s.empty()) { return; }
    std::cout << s[0]; // BEFORE
    printForward(s.substr(1));
}
```

```
// prints "cba"
void printReverse(std::string s) {
    if (s.empty()) { return; }
    printReverse(s.substr(1));
    std::cout << s[0]; // AFTER
}
```

- **Before** the recursive call: work happens on the way *down*, in the original order.
- **After** the recursive call: work happens on the way *back up*, in reverse order.
- Same idea as slip question 1: `show(3)` prints 3 2 1 1 2 3. Try it: what if a print appears both before *and* after the call?

Calls Inside Calls: printForward("abc")



Output: abc (print comes BEFORE the recursive call)

A new call pushes a frame; a return pops it.

Each call prints its first character **before** it calls the next one. Reading the red numbers: a, then b, then c, then the base case. The output grows as we go *down*.

Trace printForward("abc"): Step 1 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Call stack

printForward("abc")

Yellow: the lines being executed.

Output: a

printForward("abc") starts. s is not empty, so it prints **a**. Then it calls printForward("bc") and **waits** for that call to finish.

Trace printForward("abc"): Step 2 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack

printForward("bc")

printForward("abc")

Output: ab

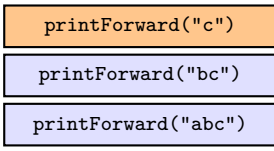
In printForward("bc"): print **b**, then call printForward("c").
The first call is still waiting below.

Trace printForward("abc"): Step 3 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack



Output: abc

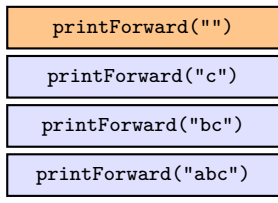
In `printForward("c")`: print **c**, then call `printForward("")`. Three calls are waiting.

Trace printForward("abc"): Step 4 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack



Output: abc

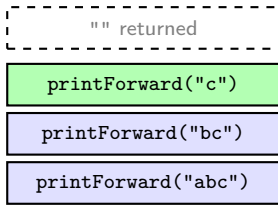
`printForward("")`: `s` is empty. This is the **base case**: print nothing and return.

Trace printForward("abc"): Step 5 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack



Output: abc

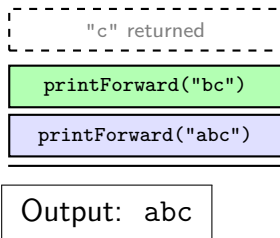
`printForward("")` returns to `printForward("c")`. Nothing is left to do after the call (the print was *before* it), so `printForward("c")` returns too.

Trace printForward("abc"): Step 6 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack



`printForward("c")` returns to `printForward("bc")`, which has nothing left to do and returns.

Trace printForward("abc"): Step 7 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Yellow: the lines being executed.

Call stack

"bc" returned

printForward("abc")

Output: abc

printForward("bc") returns to printForward("abc"), which returns.

Trace printForward("abc"): Step 8 of 8

```
void printForward(std::string s) {  
    if (s.empty()) { return; }  
    std::cout << s[0];  
    printForward(s.substr(1));  
}
```

Call stack

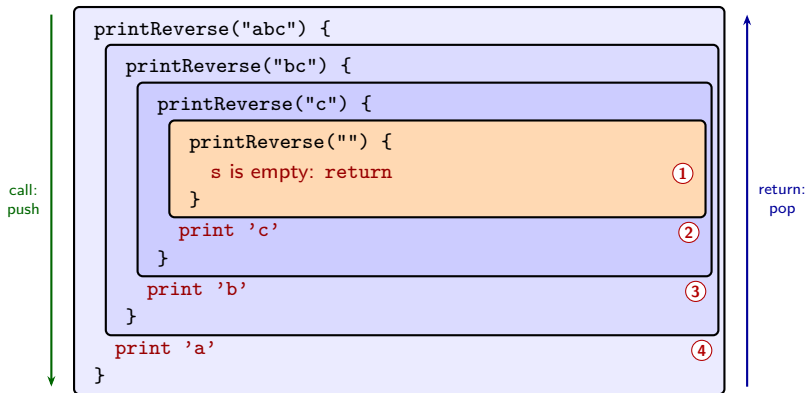
"abc" returned

Yellow: the lines being executed.

Output: abc

`printForward("abc")` returns to its caller. The stack is empty. The output is **abc**: every print happened on the way *down*; the way back up did nothing.

Calls Inside Calls: printReverse("abc")



Output: `cba` (print comes AFTER the recursive call)

A new call pushes a frame; a return pops it.

Each call **first** calls the next one and prints only **after** that call comes back. The base case ends first (1), then c (2), b (3), a (4). The output grows as we come back *up*.

Trace printReverse("abc"): Step 1 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Call stack

printReverse("abc")

Yellow: the line being executed.

Output: (nothing yet)

printReverse("abc") starts. s is not empty, so it calls printReverse("bc") **first**. The print line is still waiting below that call.

Trace printReverse("abc"): Step 2 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack

printReverse("bc")

printReverse("abc")

Output: (nothing yet)

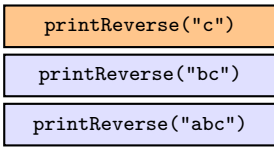
printReverse("bc") also calls first: printReverse("c"). Nothing has been printed yet.

Trace printReverse("abc"): Step 3 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack



Output: (nothing yet)

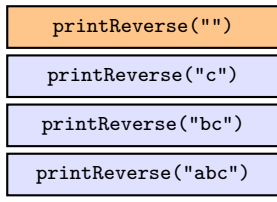
`printReverse("c")` calls `printReverse("")`. Three calls are waiting, and still no output.

Trace printReverse("abc"): Step 4 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack



Output: (nothing yet)

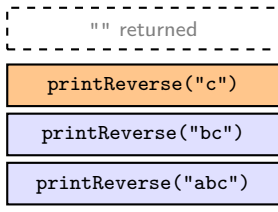
`printReverse("")`: `s` is empty. **Base case**: return immediately.

Trace printReverse("abc"): Step 5 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack



Output: **c**

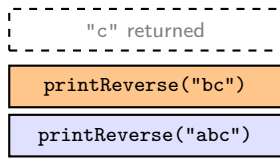
`printReverse("")` returns to `printReverse("c")`. Now it runs the line *after* the call: it prints **c**.

Trace printReverse("abc"): Step 6 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack



Output: **c****b**

`printReverse("c")` finishes and returns. `printReverse("bc")` resumes and prints **b**.

Trace printReverse("abc"): Step 7 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Yellow: the line being executed.

Call stack

"bc" returned

printReverse("abc")

Output: cb**a**

printReverse("bc") returns. printReverse("abc") resumes and prints **a**.

Trace printReverse("abc"): Step 8 of 8

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }  
    printReverse(s.substr(1));  
    std::cout << s[0];  
}
```

Call stack

Yellow: the line being executed.

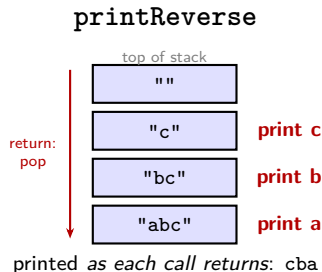
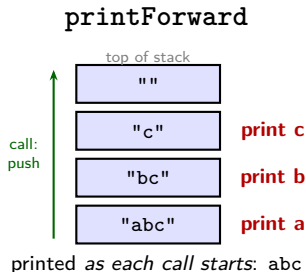
"abc" returned

Output: cba

`printReverse("abc")` returns to its caller. The stack is empty.
Output: **cba**. Every print happened on the way *back up*.

Same Stack, Different Moment to Print

Both functions make the same four calls and use the same stack. Only the moment of the print differs.



Another Example: Sum of the Digits

`digitSum(472)` should return $4 + 7 + 2 = 13$.

```
int digitSum(int n) {           // n >= 0
    if (n < 10) { return n; } // base case: one digit
    return n % 10 + digitSum(n / 10); // last digit + the rest
}
```

Call	Last digit $n \% 10$	Returns
<code>digitSum(472)</code>	2	$2 + \text{digitSum}(47) = 2 + 11 = 13$
<code>digitSum(47)</code>	7	$7 + \text{digitSum}(4) = 7 + 4 = 11$
<code>digitSum(4)</code>	(base case)	4

Same three parts

Base case: one digit. **Smaller problem:** $n / 10$ drops the last digit.

Combine: add the digit we removed.

Trace digitSum(472): Step 1 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(472) 2 + ?

Value handed back: (none yet)

digitSum(472): n is not below 10. The last digit is 2. It must compute $2 + \text{digitSum}(47)$, so it calls digitSum(47) and **waits**.

Trace digitSum(472): Step 2 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(47) 7 + ?

digitSum(472) 2 + ?

Value handed back: (none yet)

digitSum(47): last digit 7. It needs 7 + digitSum(4), so it calls digitSum(4) and waits. Two calls are waiting.

Trace digitSum(472): Step 3 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(4) base case
digitSum(47) 7 + ?
digitSum(472) 2 + ?

Value handed back: (none yet)

digitSum(4): $n < 10$. This is the **base case**: return 4. No more calls.

Trace digitSum(472): Step 4 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(4) returned 4

digitSum(47) $7 + 4 = 11$

digitSum(472) $2 + ?$

Value handed back: **4**

digitSum(4) returns **4** to digitSum(47). Now it can finish $7 + 4 = 11$.

Trace digitSum(472): Step 5 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(47) returned 11

digitSum(472) $2 + 11 = 13$

Value handed back: **11**

digitSum(47) returns **11** to digitSum(472). Now it can finish $2 + 11 = 13$.

Trace digitSum(472): Step 6 of 6

```
int digitSum(int n) {  
    if (n < 10) { return n; }  
    return n % 10 + digitSum(n / 10);  
}
```

Yellow: the line being executed.

Call stack

digitSum(472) returned 13

Value handed back: **13**

digitSum(472) returns **13** to its caller. The stack is empty. Unlike printForward, the work (the addition) happens on the way *back up*, after each call returns.

A Recipe for Any New Recursion Problem

A useful recipe for most recursion problems. Ask these questions, in this order:

- ❶ **What is the smallest version I can answer immediately?**
That is the base case.
- ❷ **What is a slightly smaller version of the same problem?**
(shorter string, smaller number, fewer cells left)
- ❸ **Leap of faith:** if the recursive call already gives the right answer for that smaller problem, what is left to do?
- ❹ **Does every call get closer to the base case?** If not, the stack overflows.

Practice at home

- (1) `countDown(n)` prints n down to 1; then `countUp(n)` prints 1 up to n . Which line moves?
- (2) `power(x, n)` returns x^n using $x^n = x \cdot x^{n-1}$.
- (3) `countChar(s, ch)`: how many times does `ch` appear in `s`?

Your Turn: What Is the Signature of the Helper?

Often the function the user calls is not the function that recurses. The public function stays simple; a private **helper** carries the extra information the recursion needs.

(a) The user calls `void printReverse(const std::string& s)`. We want a helper that avoids copying the string with `substr`. It should print `s` backward by moving an index. Write the helper's signature.

(b) The user calls `bool reachable(const Maze& m, Coord start, Coord goal)`. The recursive search needs to remember which cells it has already visited. Write the helper's signature.

Ask yourself

1. What changes from one call to the next? (*parameter*) 2. What stays the same? (*const reference*) 3. What must every call share and update? (*non-const reference*) 4. What does the caller need back? (*return type*)

Answers: Signature of the Helper (a)

The original question

(a) For `void printReverse(const std::string& s)`, write a helper that prints `s` backward using an index, not `substr`.

```
void helper(const std::string& s, size_t i) { // i = current index
    if (i == s.size()) { return; }           // base case
    helper(s, i + 1);                         // smaller problem
    std::cout << s[i];                       // print AFTER
}
void printReverse(const std::string& s) { helper(s, 0); }
```

The index `i` *changes* (parameter by value); `s` *stays the same* (const reference, so no copy). The helper comes *first*, so the compiler has seen it when `printReverse` calls it.

Why it matters: efficiency

Each `substr` copies the rest of the string: about $n^2/2$ characters copied in all (quadratic). The index version copies nothing: n steps (linear).

Answers: Signature of the Helper (b)

The original question

(b) The user calls `bool reachable(const Maze& m, Coord start, Coord goal)`. The recursive search must remember which cells it has visited. Write the helper's signature.

```
bool helper(const Maze& m, Coord cur, Coord goal,  
            std::vector<std::vector<bool>>& visited);
```

- `cur` *changes* with each call (by value).
- `m` is a `const` reference, and `goal` is unchanged during the search.
- `visited` must be shared and updated by every call, so it is a **non-const reference**: a copy would forget marks made by deeper calls.
- It returns `bool`: “is there a path from `cur`?”

DFS in General: One Slide

The picture. Explore a maze with chalk and a ball of string:

- Walk as far as you can. Chalk-mark every cell you enter.
- At a **dead end**, follow the string back to the last cell that still has an unexplored direction. That is **backtracking**.
- **Never rub out chalk.** A marked cell is never entered again.

The recursion. Each call asks:

“Starting from this cell, is there a path to the target?”

- Mark the cell visited.
- **Base case:** it is the target: success.
- Otherwise try each neighbor in a fixed order (left, right, up, down). If a neighbor is open and unvisited, call the same function on it.
- If any call succeeds, this call succeeds. If all fail, return failure.

The **call stack** is the ball of string: its frames are the cells on the current route.

Slip Question 3: The Setup

	0	1	2
0	S		
1			
2	T		

$S = (0, 0)$, $T = (2, 0)$. Black cells are walls. Cells are written (row, column).

A call marks its cell visited, then tries its neighbors in the order **left, right, up, down**, recursing into any that is open and unvisited.

- (a) Order in which calls are entered?
- (b) Dead end at (2, 2): which calls return, and where does DFS resume?
- (c) Final route, and why is it not on the first branch?

Orange: current call. Blue: visited, call still active. Gray: visited, returned empty-handed. Green: final route.

Slip Question 3, Step 1 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,0)

Answer stack

(empty)

Start: call `helper(0,0)`. Mark it visited. Neighbors: left outside, right **open**. Go there next.

Slip Question 3, Step 2 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,1)`. Mark it visited. Neighbors: left visited, right **open**. Go there next.

Slip Question 3, Step 3 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,2)`. Mark it visited. Neighbors: left visited, right outside, up outside, down **open**. Go there next.

Slip Question 3, Step 4 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(1,2)
helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(1,2)`. Mark it visited. Neighbors: left wall, right outside, up visited, down **open**. Go there next.

Slip Question 3, Step 5 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(2,2)
helper(1,2)
helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(2,2)`. Mark it visited. Neighbors: left wall, right outside, up visited, down outside. **Dead end**: return.

Slip Question 3, Step 6 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,0)

Answer stack

(empty)

helper(2,2), helper(1,2), helper(0,2), helper(0,1) returned empty-handed (backtracking), each leaving the stack. Back in helper(0,0). Remaining neighbors: up outside, down **open**. Go there next.

Slip Question 3, Step 7 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(1,0)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(1,0)`. Mark it visited. Neighbors: left outside, right wall, up visited, down **open**. Go there next.

Slip Question 3, Step 8 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(2,0)
helper(1,0)
helper(0,0)

Answer stack

(2,0)

Move down: call `helper(2,0)`. Mark it visited. It is the **target** (base case): push it on the answer stack and return success.

Slip Question 3, Step 9 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

(no calls left)

Answer stack

(0,0)
(1,0)
(2,0)

Success unwinds the call stack: each frame sees a non-empty answer stack, pushes its own cell and returns. The route is $(0,0) \rightarrow (1,0) \rightarrow (2,0)$. The cells on the first branch were *visited* but are not on the route.

Slip Question 3: Answer (a)

The problem. $S = (0, 0)$, $T = (2, 0)$; black cells are walls; cells are (row, column). Each call marks its cell visited, then tries its neighbors in the order **left, right, up, down**.

	0	1	2
0	S 1	2	3
1	6		4
2	T 7		5

Numbers: the order in which calls are entered.

The original question

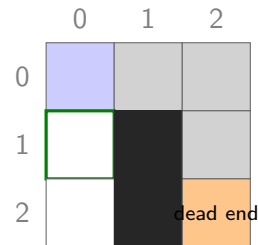
(a) In what order are the calls entered?

Answer.

$(0, 0)$, $(0, 1)$, $(0, 2)$, $(1, 2)$, $(2, 2)$, $(1, 0)$, $(2, 0)$: seven calls. DFS tries *right* before *down*, so it first follows the whole top-and-right branch down to $(2, 2)$, and only then goes down from $(0, 0)$.

Slip Question 3: Answer (b)

The problem. $S = (0,0)$, $T = (2,0)$; black cells are walls; cells are (row, column). Each call marks its cell visited, then tries its neighbors in the order **left, right, up, down**.



Orange: dead end. Gray: returned empty-handed. Green outline: where DFS resumes.

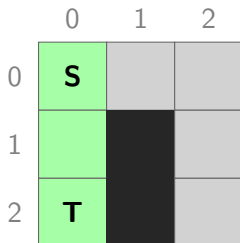
The original question

(b) The search reaches the dead end at $(2,2)$. Which calls return, and where does DFS resume?

Answer. At $(2,2)$: left is a wall, up is visited, right and down are outside, so it returns empty-handed to $(1,2)$. Then $(1,2)$, $(0,2)$, $(0,1)$ each return, since none has another open, unvisited neighbor. At $(0,0)$: up is outside, and down, $(1,0)$, is open and unvisited, so DFS **resumes at** $(1,0)$.

Slip Question 3: Answer (c)

The problem. $S = (0, 0)$, $T = (2, 0)$; black cells are walls; cells are (row, column). Each call marks its cell visited, then tries its neighbors in the order **left, right, up, down**.



Green: on the final route. Gray: visited, but not on the route.

The original question

(c) What is the final route, and why is it not on the first branch?

Answer. Route: $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0)$. DFS tries *right* before *down*, and the right-hand branch ends in a dead end: its cells were *visited*, but they are not on the route.

Key distinction

Visited means “a call entered this cell”. **On the route** means “a call below it reached the target”. Seven cells were visited; three are on the route.

Each Call's Choice Between Its Neighbors

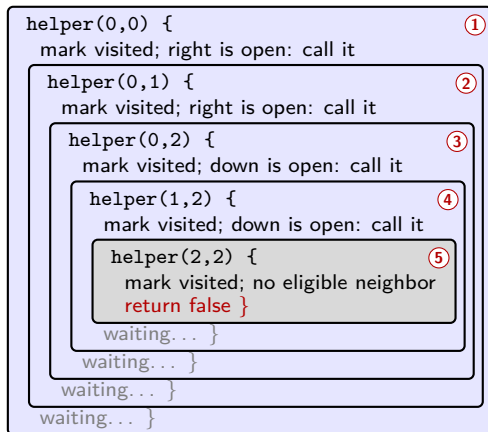
Call	left	right	up	down	Outcome
(0,0)	outside	open: call (0,1)	outside	open: call (1,0)	success through down
(0,1)	visited	open: call (0,2)	outside	wall	fails
(0,2)	visited	outside	outside	open: call (1,2)	fails
(1,2)	wall	outside	visited	open: call (2,2)	fails
(2,2)	wall	outside	visited	outside	dead end , fails
(1,0)	outside	wall	visited	open: call (2,0)	success
(2,0)	target: base case				success

- A neighbor is skipped for exactly three reasons: **outside** the grid, a **wall**, or **already visited**.
- (0,1)'s only open neighbor leads nowhere, so it fails too. A call fails when every neighbor was skipped or failed.
- Only (0,0) has *two* calls to make: right (failed), then down (succeeded).

Slip Question 3 as Nested Calls: the First Branch

	0	1	2
0	1	2	3
1			4
2			5

Numbers: order of the calls.
Blue: call still active. Orange:
current call, a dead end.



Each call marks its cell and calls the next open neighbor *inside* itself, like `printForward`. At (2,2) no neighbor is both open and unvisited ((1,2) is open but already visited), so call 5 returns false.

Slip Question 3 as Nested Calls: the Second Branch

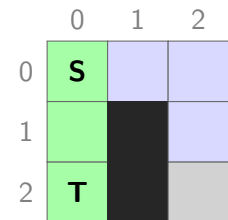
	0	1	2
0	1		
1	6		
2	7		

Gray: visited, returned false.
Green: on the route. Numbers:
order of the calls.

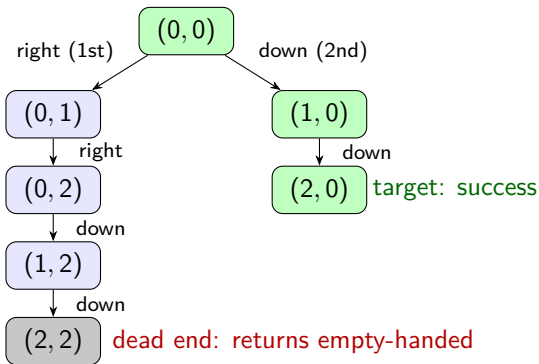
```
helper(0,0) { ①  
  mark visited  
  helper(0,1) { ... }  
    returned false (visited, no route)  
  helper(1,0) { ⑥  
    mark visited; down is open: call it  
    helper(2,0) { ⑦  
      this is the target T  
      return true }  
    return true }  
  return true }
```

Back in call 1: the right-hand branch returned false, so it tries down.
Call 6, (1,0), calls call 7, (2,0), the target. True comes back up: the route is (0,0) → (1,0) → (2,0).

The Same Search as a Call Tree



Green: on the route. Blue: visited, dead-end branch. Gray: the dead end (2, 2).



Each node is one call (one stack frame). Going down an edge is a call; coming back up is a return. The left branch is explored completely first, and *only then* does (0,0) try its second choice.

Does the Neighbor Order Matter?

Question: on slip question 3, must I try left and right before down, or can I go straight down?

As the slip says

Left, right, up, down: 7 calls, $(0, 0), \dots, (2, 2), (1, 0), (2, 0)$. Route $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0)$.

Down first (not asked)

3 calls, $(0, 0), (1, 0), (2, 0)$. Same route here.

- Any fixed order is a legitimate DFS: it finds a path whenever one exists, but may enter cells in a different order and find a different route.
- So the order is **part of the question**: follow the order the problem states. Slip question 3 says left, right, up, down, so the answer to (a) is the 7-cell list.
- For Project 2, follow the neighbor order specified in your handout.

When We Backtrack: What Is Undone?

Question: do we unmark the cells we used? Do we remove them from stack memory?

Thing	Result	Why
Visited mark of the cell	Stays marked	For this DFS, once a cell has been explored we do not need to explore it again. Un-marking could make the search explore the same cells and dead ends repeatedly.
The call's stack frame	Removed	Returning from a call pops its frame automatically. You do not write any code for this.
Answer stack	Unchanged	Only calls that were followed by success push their cell. Dead-end cells never push.

In one sentence

In this reachability DFS, an unsuccessful call simply **returns**; the visited marks remain and the frames disappear by themselves. Other backtracking algorithms (such as the Make 24 project) may also **undo changes** to shared state.

BFS vs. DFS

	DFS	BFS
Idea	Go as deep as possible along one branch, then back up	Explore all cells at distance 1, then 2, then 3, ...
Container	Stack (the call stack, or an explicit one)	Queue
Route found	The first route it finds: not necessarily the shortest	A route with the fewest cells
Slip maze	Enters 7 cells before reaching T	Stop at dequeue: 5 dequeued, 6 discovered
Project 2 example (4×4)	Route of 9 cells	Route of 7 cells

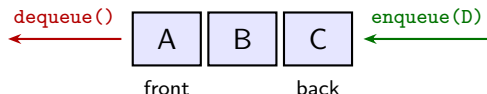
Note. Both counts use the same neighbor order. The BFS count stops when T is *dequeued*. Class 12 stops when T is *discovered* (enqueued): 3 dequeued, 5 discovered. The route is the same. BFS guarantees the fewest moves when every move costs the same.

Next class: we write BFS with an explicit queue and compare the two on the same maze.

What Is a Queue? (and “enqueue”)

I read the question as “what is a queue (enqueue / dequeue)?” If you meant something else, ask me in class.

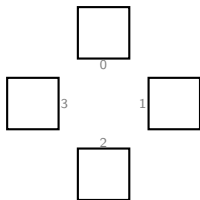
A **queue** is a line at a store: the first person to join is the first person served. We say **FIFO**: first in, first out.



- **enqueue(x)**: add x at the back. **dequeue()**: remove and return the item at the front.
- A **stack** is the opposite: last in, first out (a pile of plates).
- Why we care today: DFS uses a stack (the call stack); BFS next class uses a **queue**. Class 10 covered queues in detail.

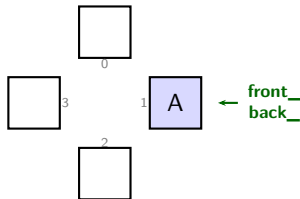
Counting Elements: Empty and One Element

Empty



$\text{front_} = \text{back_} = -1$
 $\text{size} = 0$

One element

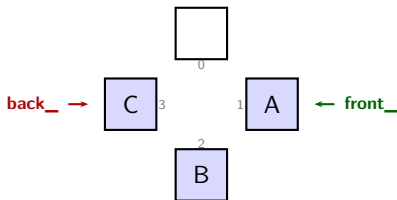


$\text{front_} = \text{back_} = 1$
 $\text{size} = 1 - 1 + 1 = 1$

CAPACITY = 4. front_ is the index of the *first* element and back_ the index of the *last*. An empty queue has no first or last element, so both are -1 and the size is 0. With exactly one element, both indices name the *same* cell, and $\text{back_} - \text{front_} + 1 = 1$.

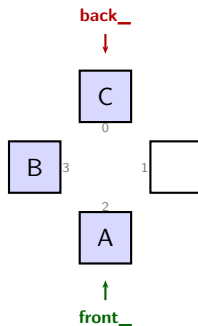
Counting Elements: Not Wrapped and Wrapped

Not wrapped



$$\text{front_} = 1, \text{back_} = 3$$
$$3 - 1 + 1 = 3$$

Wrapped

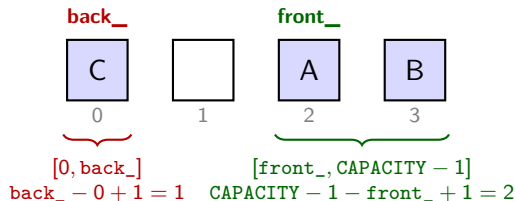


$$\text{front_} = 2, \text{back_} = 0$$
$$4 - (2 - 0 - 1) = 3$$

Not wrapped ($\text{back_} \geq \text{front_}$): cells $\text{front_} \dots \text{back_}$ are used:
 $\text{back_} - \text{front_} + 1$. **Wrapped** ($\text{back_} < \text{front_}$): the queue runs past the end and continues at cell 0. Only the $\text{front_} - \text{back_} - 1$ cells between `back_` and `front_` are empty (here cell 1), so $\text{size} = \text{CAPACITY} - (\text{front_} - \text{back_} - 1)$.

Wrapped Queue: A Second Way to Count

Cut the wrapped queue into two pieces that are each *not* wrapped (CAPACITY = 4, front_ = 2, back_ = 0):



Add the two pieces

$(\text{back_} + 1) + (\text{CAPACITY} - \text{front_})$. Here: $(0 + 1) + (4 - 2) = 3$.

Same as the first formula:

$\text{CAPACITY} - (\text{front_} - \text{back_} - 1) = \text{CAPACITY} - \text{front_} + \text{back_} + 1$. Count the *gap* and subtract, or count the *two pieces* and add.

Counting Elements: The Rule and the Code

Case	Test	Size
Empty	<code>front_ == -1</code>	0
Not wrapped	<code>back_ >= front_</code>	<code>back_ - front_ + 1</code>
Wrapped	<code>back_ < front_</code>	<code>CAPACITY - (front_ - back_ - 1)</code>

Examples with `CAPACITY = 4`, as $(\text{front}_-, \text{back}_-)$: $(-1, -1) \rightarrow 0$, $(2, 2) \rightarrow 1$, $(1, 3) \rightarrow 3$, $(2, 0) \rightarrow 3$, $(1, 0) \rightarrow 4$.

```
int size() const {
    if (front_ == -1) { return 0; }           // empty
    if (back_ >= front_) { return back_ - front_ + 1; } // not wrapped
    return CAPACITY - (front_ - back_ - 1);   // wrapped
    // same value: back_ + 1 + (CAPACITY - front_)
}
```

- Why subtract? The array has `CAPACITY` cells. In the wrapped case, every cell is either in the queue or in the gap between `back_` and `front_`.
- Check the full wrapped queue $(1, 0)$: $\text{gap} = 1 - 0 - 1 = 0$, so $\text{size} = 4 - 0 = 4$.

What Is `std::string::size_type`?

It is the **unsigned integer type** that `std::string` uses for lengths and positions. `s.length()` returns it. On a typical 64-bit system it is the same as `std::size_t` (an 8-byte unsigned integer).

```
void printReverse(const std::string& s,
                  std::string::size_type i) {
    if (i == s.length()) { return; } // same type on both sides
    printReverse(s, i + 1);
    std::cout << s[i];
}
```

- We use it for `i` so that `i == s.length()` compares two values of the *same* kind (no signed/unsigned warning).
- You may write `int i` in simple code, but the compiler may warn when you compare an `int` with `s.length()`.
- **Careful:** unsigned values never go below 0. For an empty string, `s.length() - 1` is not `-1`: it wraps to a huge number (18446744073709551615 on a typical 64-bit system).

Reviewing Arrays, Stacks, and Queues

Reviewing is a good plan: recursion builds directly on them.

- **Stack** (Class 9): the **call stack** behaves like a stack. A call pushes a frame; a return pops it; the newest call finishes first.
- **Queue** (Class 10): BFS next class uses a queue, so the circular-buffer ideas return.
- **Where to review:** the Class 9 and Class 10 slides, the Class 10 Q&A deck, the Class 10 and Class 11 flashcards, and the Queue and Recursion Card Quest games on the schedule page.

A 10-minute routine

Do one card-game round, then redo slip questions 1 and 3 from memory, then compare with the key.

Take-Aways

- 1 Every recursion needs a **base case** and a **smaller problem**; any remaining work happens before the call, after it (a **combine** step), or not at all. Printing *before* the call goes forward; printing *after* it goes backward.
- 2 DFS = “is there a path from here?” Mark the cell, try the neighbors in the stated order, and return success or failure.
- 3 In this DFS, backtracking is just **returning**: frames pop, **visited marks stay**. (Other backtracking algorithms may also undo choices.)
- 4 **Visited** is not the same as **on the route**.
- 5 DFS uses a **stack**; BFS (next class) uses a **queue**.

Keep asking questions on the slips: they shape the next class.